

v1.9 administration/configuration specification

Yes. v1.9 is an important architectural step because it turns the capabilities introduced in v1.5–v1.8 into a **configurable legal-practice platform** rather than a system whose behavior is hard-coded.

ILCMS — Technical Specification v1.9

Administration, Configuration, Master Data & System Management Specification

Project: Icelandic Legal Case Management System (ILCMS)

Specification Version: 1.9

Status: Technical Design Baseline

Depends on: v1.1–v1.8

Primary Areas: Administration, Configuration, Master Data, Organizations, Users, System Management

1. Purpose

This specification defines the administration and configuration architecture for ILCMS.

The purpose is to ensure that organizations can configure the system without modifying application source code.

Configuration includes:

- organizations;
- departments and teams;
- users;
- roles and permissions;
- case types;
- classifications;
- workflows;
- task templates;
- deadline rules;
- calendars;
- notification policies;
- document categories;

- retention policies;
- custom fields;
- numbering schemes;
- system settings;
- integrations;
- localization;
- reporting configuration;
- feature flags.

The administration subsystem must provide controlled configuration while preventing administrators from accidentally changing historical legal records or compromising security.

2. Design Principles

2.1 Configuration is data

Where behavior can reasonably be expressed as configuration, it should be stored as configuration rather than hard-coded.

Examples:

Case types

Workflow states

Task templates

Deadline rules

Notification rules

Document categories

Classification structures

2.2 Configuration must be versioned

Configuration affecting active cases should normally be versioned.

For example:

Civil Case Workflow v1

Civil Case Workflow v2

Civil Case Workflow v3

A new workflow version should not silently modify the behavior of existing cases.

2.3 Historical records are immutable

Configuration changes must not rewrite history.

For example, changing:

Case type name:

"Employment"

to:

"Employment Dispute"

must not change historical audit records in a way that destroys the original meaning.

Where historical labels are important, snapshots or version references should be retained.

2.4 Security configuration is especially protected

Administrative configuration must not provide an indirect method to bypass v1.5 authorization.

Changing configuration must itself require authorization.

Security-sensitive changes should require:

- elevated privileges;
- audit logging;
- optional approval;
- strong authentication.

3. Administrative Roles

The system should distinguish between different administrative responsibilities.

Recommended roles:

SYSTEM_ADMIN

ORGANIZATION_ADMIN

SECURITY_ADMIN

CASE_ADMIN

DOCUMENT_ADMIN

WORKFLOW_ADMIN

REPORT_ADMIN

INTEGRATION_ADMIN

AUDITOR

These are logical roles.

Actual permissions should be defined through the authorization model from v1.5.

4. System Administrator

A `SYSTEM_ADMIN` manages platform-level configuration.

Responsibilities may include:

- system configuration;
- deployment settings;
- global feature flags;
- system integrations;
- global security policies;
- tenant/organization creation;
- system health;
- job management;
- system-wide templates.

A system administrator should not automatically have unrestricted access to confidential legal case content.

This distinction is important:

`System administration`

$\neq$

`Legal case access`

5. Organization Administrator

An `ORGANIZATION_ADMIN` manages a legal organization.

Responsibilities:

- departments;
- teams;
- users;
- organizational roles;
- case configuration;

- workflows;
- document categories;
- notification policies;
- organizational calendars;
- reports.

Organization administrators must be restricted to their organization.

6. Security Administrator

Security administrators manage security configuration such as:

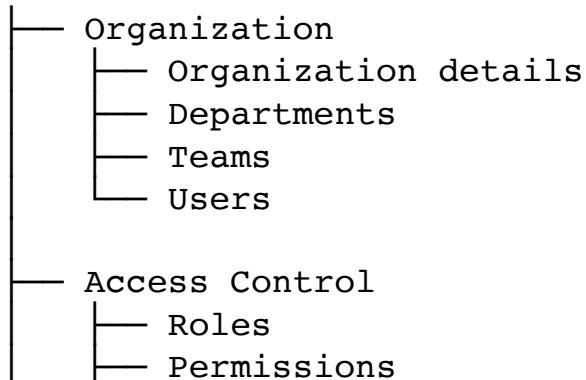
- authentication policies;
- MFA requirements;
- session policies;
- password policy where local authentication exists;
- API credentials;
- service accounts;
- access policies;
- security monitoring.

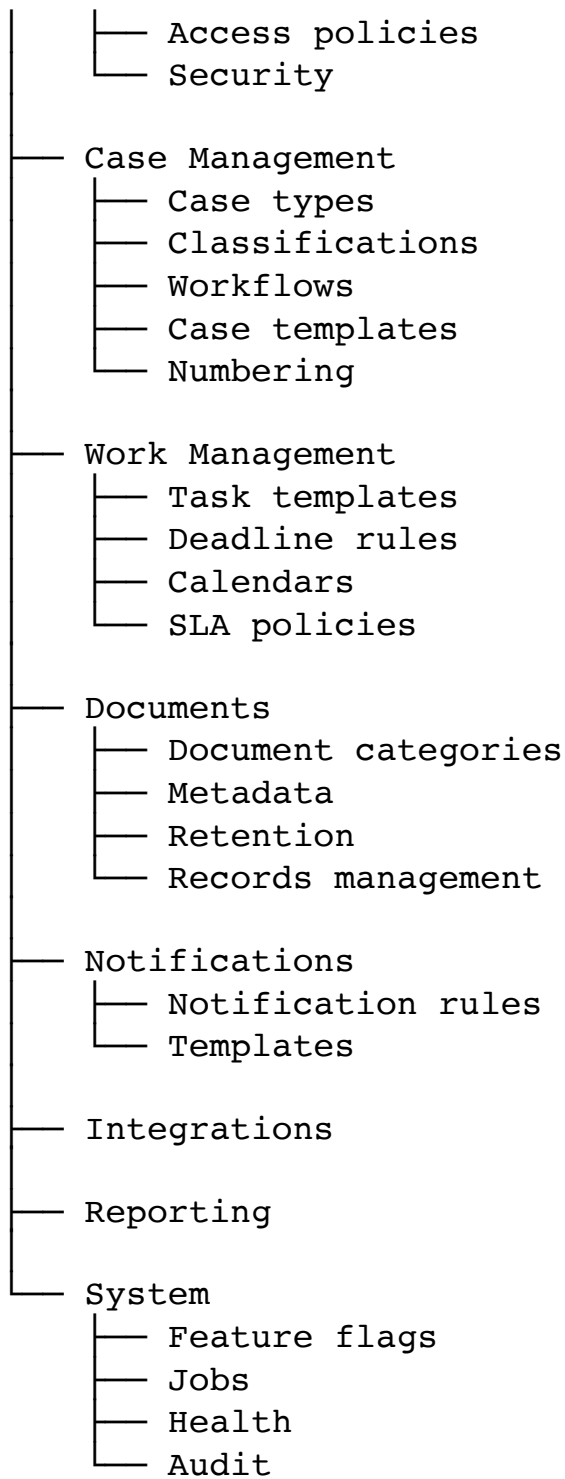
Security administrators should not automatically gain access to case content.

7. Administrative UI

The administration interface should be organized into sections:

Administration

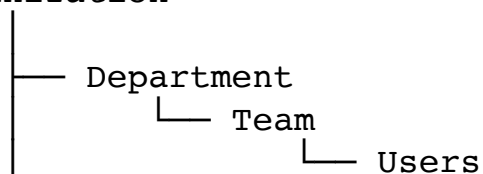


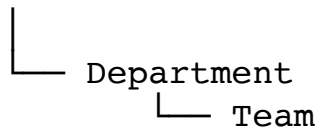


8. Organization Model

The organizational hierarchy should support:

Organization





Users may belong to multiple teams.

9. Organization Fields

Suggested organization model:

`organizations`

`id`

`name`

`legal_name`

`organization_code`

`description`

`country`

`default_timezone`

`default_locale`

`default_currency`

`status`

`created_at`

`updated_at`

Recommended status values:

ACTIVE

SUSPENDED

DEACTIVATED

10. Organization Isolation

All organization-scoped configuration must include:

`organization_id`

Examples:

- workflows;
- case types;
- classifications;
- notification rules;
- templates;
- reports;

- custom fields.

Cross-organization configuration should only be possible through explicitly authorized system-level functionality.

11. Departments

Suggested model:

```
departments
-----
id
organization_id
name
code
description
manager_user_id
status
created_at
updated_at
```

Departments are organizational structures rather than authorization boundaries unless explicitly configured as such.

12. Teams

Suggested model:

```
teams
-----
id
organization_id
department_id
name
code
description
manager_user_id
status
created_at
updated_at
```

Teams may be used for:

- case assignment;
- task assignment;

- workload reporting;
- access control;
- notifications.

13. User Administration

User records should normally originate from the identity system.

ILCMS should maintain an internal profile:

```
users
-----
id
identity_provider
external_subject
username
display_name
email
status
locale
timezone
created_at
updated_at
```

The external identity remains authoritative for authentication.

14. User Lifecycle

User lifecycle:

```
INVITED
  ↓
ACTIVE
  ↓
SUSPENDED
  ↓
DEACTIVATED
```

Deactivation must immediately prevent normal authentication and access.

Historical assignments must remain intact.

For example:

Task assigned to User A

User A leaves organization

↓

User A deactivated

↓

Historical task remains assigned to User A
The system may provide reassignment tools but must not rewrite history.

15. User-Team Membership

Suggested table:

team_memberships

id

team_id

user_id

role

joined_at

removed_at

created_by

Membership history must be retained.

16. Role Management

Roles should be configurable within controlled boundaries.

Example:

ROLE

↓

Permissions

↓

Resources/actions

Example:

CASE_EDITOR

cases.read

cases.update

tasks.read

tasks.update

17. Permission Model

Permissions should use stable machine-readable identifiers.

Examples:

```
cases.read
cases.create
cases.update
cases.close
cases.reopen
```

```
documents.read
documents.create
documents.update
documents.delete
```

```
tasks.read
tasks.create
tasks.update
tasks.complete
```

```
deadlines.read
deadlines.create
deadlines.update
```

```
reports.read
reports.export
```

```
admin.users
admin.workflows
admin.security
```

18. Permission Boundaries

The permission system should distinguish:

Action permission
from:

Resource access
Having:

```
cases.read
```

does not mean a user can read every case.

Both must succeed:

Permission check

+
Object authorization
↓
Access granted

19. Case Type Administration

Case types should be configurable.

Examples:

CIVIL
CRIMINAL
FAMILY
EMPLOYMENT
CORPORATE
CONTRACT
PROPERTY
INSURANCE
IMMIGRATION
ADMINISTRATIVE
DEBT
ESTATE
OTHER

Each case type may reference:

- workflow;
- classification;
- default tasks;
- default deadlines;
- required fields;
- document categories;
- default team;
- notification rules.

20. Case Type Model

Suggested table:

case_types

id
organization_id
code
name
description
active
workflow_id
default_classification_id
created_at
updated_at
Codes must be stable.

Names may be localized.

21. Classification Administration

The classification system must support the records-management model established in v1.6.

Example:

Legal Matters

- Civil
 - Contracts
 - Property
 - Damages
- Employment
 - Termination
 - Employment disputes

The system should support hierarchical classifications.

22. Classification Versioning

Classification changes must preserve historical references.

Possible model:

classification_set
classification_version
classification_node

A case should retain the classification version under which it was created or classified.

23. Workflow Administration

Workflow administration from v1.7 must be exposed through a controlled UI.

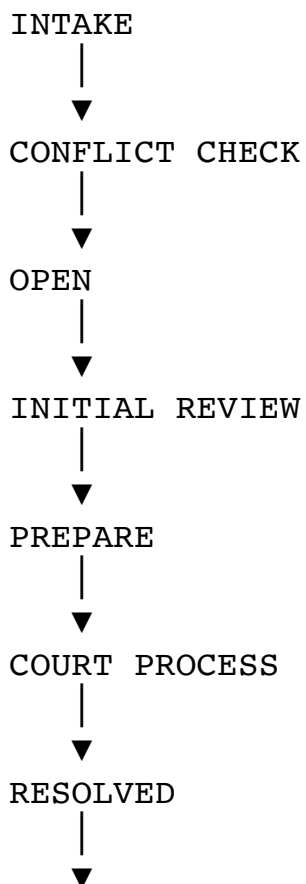
Administrators can define:

Workflow
↓
States
↓
Transitions
↓
Tasks
↓
Deadlines
↓
Notifications

24. Workflow Designer

A visual workflow designer may be provided.

Example:



CLOSED

The visual designer is a representation of the authoritative workflow definition.

25. Workflow Validation

Before activation, a workflow must be validated.

Validation should detect:

- unreachable states;
- missing initial state;
- invalid transitions;
- circular dependencies where prohibited;
- missing required fields;
- invalid task references;
- invalid deadline rules;
- duplicate state codes.

An invalid workflow cannot be activated.

26. Workflow Version Lifecycle

DRAFT



VALIDATED



PUBLISHED



ACTIVE



RETIRED

Published configuration should not be directly modified.

Changes create a new version.

27. Case Templates

Case templates simplify case creation.

A template may define:

case type
classification
workflow
default team
required fields
default tasks
default deadlines
document categories
notification rules
Example:

Employment Dispute Template

↓

Creates:

Case type = EMPLOYMENT
Workflow = Employment v4
Classification = Employment/Disputes
Tasks = Initial review, Evidence collection
Documents = Contract, Correspondence

28. Task Template Administration

Suggested model:

```
task_templates
-----
id
organization_id
name
description
case_type_id
workflow_state_id
priority
relative_due_rule
assigned_role
active
version
```

Tasks generated from templates must retain the originating template/version.

29. Deadline Rule Administration

Deadline rules are high-impact configuration and require elevated authorization.

A rule may define:

Trigger event

+

Start date

+

Calendar

+

Offset

+

Calculation method

+

Result

Example:

Trigger:

Document received

Start:

received_at

Offset:

14 days

Calendar:

LEGAL_WORKING_DAYS

Result:

Deadline

30. Deadline Rule Validation

Deadline rules must be independently testable.

Administrators should be able to enter:

Trigger date

Calendar

Rule

and receive:

Calculated deadline

before publishing the rule.

The calculation should show the inputs and intermediate assumptions where appropriate.

31. Deadline Rule Versioning

Never silently change a deadline rule used by active cases.

Example:

Rule v1 → Existing cases

Rule v2 → New cases

Migration of active cases requires an explicit operation.

32. Legal Calendars

Administrators may configure calendars for:

Icelandic public holidays

Court working days

Organization working days

Custom closures

Calendar definitions should be versioned.

33. Notification Rule Administration

Notification rules should use:

Trigger

Condition

Recipient

Channel

Template

Delay/threshold

Priority

Example:

Trigger:

Deadline approaching

Condition:

7 days remaining

Recipient:

Responsible lawyer

Channel:

In-app + email

34. Notification Templates

Notification text should be configurable and localized.

Example:

Subject:
Upcoming deadline

Body:
Deadline {{deadline.title}} for case {{case.case_number}}
is due on {{deadline.due_at}}.
Template rendering must be sandboxed.

User-controlled data must be safely escaped.

35. Document Category Administration

Administrators can configure document categories such as:

PLEADING
CONTRACT
COURT_DOCUMENT
EVIDENCE
CORRESPONDENCE
DECISION
JUDGMENT
INVOICE
IDENTIFICATION
OTHER

Categories may determine:

- metadata requirements;
- retention policy;
- OCR requirement;
- indexing;
- workflow;
- access restrictions.

36. Retention Policy Administration

Retention policies must integrate with v1.6.

A policy may define:

Record type
Trigger event
Retention period
Disposition action
Legal hold behavior
Approval requirement
Example:

Case closed

↓

Retention period begins

↓

Retention expires

↓

Disposition review

↓

Authorized destruction/archive

The system must never automatically destroy records merely because a configured timer expired unless the applicable policy explicitly permits automated disposition.

37. Legal Holds

Legal holds override ordinary disposition processes.

Administration must support:

Create hold
Modify hold
Release hold
View affected records
While a hold is active:

Retention expiry

↓

BLOCKED

All hold operations must be audited.

38. Custom Fields

Organizations may define custom fields for:

- cases;
- parties;
- documents;
- tasks;
- proceedings.

Supported types should include:

STRING

TEXT

INTEGER

DECIMAL

BOOLEAN

DATE

DATETIME

ENUM

MULTI_ENUM

REFERENCE

39. Custom Field Security

Custom fields may contain sensitive legal information.

Therefore:

- field-level permissions should be supported where required;
- values must be included in authorization-aware APIs;
- search indexing must respect field security;
- reports must respect field security;
- audit logging must avoid unnecessarily exposing values.

40. Numbering Configuration

Case numbering must be configurable.

Example:

M- {YEAR} - {SEQUENCE}

Result:

M-2026-00125

Possible numbering components:

YEAR

MONTH

DEPARTMENT

CASE_TYPE

SEQUENCE

The generated case number must remain unique within its defined scope.

41. Numbering Guarantees

Number generation must be transactional.

Two concurrent requests must never receive the same case number.

Numbers should normally not be reused.

Gaps may occur because of failed or cancelled transactions unless a specific legal/organizational numbering policy requires otherwise.

42. System Settings

System settings should be divided into:

Platform settings

Managed by system administrators.

Organization settings

Managed by organization administrators.

User settings

Managed by individual users.

Example:

Platform:

maximum upload size

Organization:

default timezone

User:

preferred timezone

43. Configuration Precedence

Where settings exist at multiple levels:

Platform

↓

Organization

↓

Team

↓

User

More specific configuration may override broader configuration only where explicitly permitted.

Security-critical platform policies must not be weakened by lower-level configuration.

44. Feature Flags

Feature flags should support controlled rollout.

Example:

AI_DOCUMENT_EXTRACTION

ADVANCED_SEARCH

EXTERNAL_CALENDAR_SYNC

REALTIME_COLLABORATION

SEMANTIC_SEARCH

Flags may be scoped to:

- system;
- organization;
- environment.

Feature flags must not be used as security controls.

45. Configuration Audit

Every configuration change must record:

id

organization_id

actor_user_id

resource_type
resource_id
action
old_version
new_version
timestamp
request_id
reason
Examples:

WORKFLOW_PUBLISHED
DEADLINE_RULE_CHANGED
ROLE_UPDATED
RETENTION_POLICY_CHANGED
NOTIFICATION_RULE_CHANGED

46. Configuration Approval

High-impact changes should optionally require approval.

Recommended approval candidates:

- security policies;
- roles/permissions;
- deadline rules;
- retention policies;
- classification changes;
- production workflows.

Example:

Draft
↓
Submitted
↓
Approval
↓
Published

The person submitting a change should not necessarily be permitted to approve their own change.

47. Configuration Change History

Administrators should be able to inspect:

What changed?

Who changed it?

When?

Why?

Previous value?

New value?

Who approved it?

When did it become active?

Configuration history should be immutable.

48. Configuration Rollback

Rollback should be implemented as a new configuration version rather than destructive restoration.

Example:

Workflow v4

↓

Problem discovered

↓

Create v5 based on v3

↓

Publish v5

Historical versions remain available.

49. Configuration Import/Export

Organizations should be able to export configuration.

Example format:

YAML

JSON

Configuration exports may include:

case types

workflows

task templates

deadline rules

notification rules

document categories

custom fields

They must not include:

- passwords;
- OAuth tokens;
- API secrets;
- private keys;
- case documents;
- confidential case data.

50. Configuration Promotion

Configuration should support promotion between environments.

Example:

Development

↓

Test

↓

Staging

↓

Production

Configuration should be represented declaratively where practical.

This allows GitOps-style management of approved configuration.

51. Configuration Validation

Before import or activation:

Schema validation

↓

Reference validation

↓

Security validation

↓

Business-rule validation

↓

Dry run

↓

Approval

↓

Activation

Invalid configuration must be rejected before affecting production.

52. Master Data

Master data includes stable reference information such as:

- countries;
- languages;
- currencies;
- courts;
- court types;
- document types;
- case types;
- communication types;
- task priorities;
- deadline types.

Master data should be separated from transactional records.

53. Court Master Data

Court records should support:

```
court
-----
id
name
official_name
court_type
country
location
website
active
```

Court identifiers should be stable.

The system should support Icelandic courts and future additions without requiring source-code changes.

54. Court Case Number

Court case numbers must remain separate from internal ILCMS case numbers.

Example:

ILCMS:

M-2026-00125

Court:

E-1234/2026

The system must support multiple court proceedings and court case numbers for one ILCMS case as defined in v1.7.

55. Reference Data Lifecycle

Reference values should support:

ACTIVE

INACTIVE

Inactive values must remain available for historical records but should normally not be selectable for new records.

56. Integration Administration

Administrators should be able to configure integrations such as:

Identity provider

SMTP

Calendar providers

Object storage

Search

OCR

Virus scanning

Message broker

AI services

External court systems

57. Integration Credentials

Credentials must never be stored as ordinary configuration values.

Use:

Vault
Kubernetes Secrets
External Secrets
cloud secret manager
as appropriate.

The administrative UI should display:

Configured
rather than exposing:

secret_value

58. Integration Health

Each integration should expose a health status:

CONNECTED

DEGRADED

FAILED

DISABLED

NOT_CONFIGURED

Health checks must not expose secrets.

59. Administrative Jobs

Administrators should have visibility into background jobs.

Example:

Job ID

Type

Status

Created

Started

Completed

Duration

Attempts

Error

Supported actions:

Retry

Cancel

Inspect

Administrative actions must be authorized and audited.

60. Data Maintenance

Administrative maintenance tools may include:

- search reindex;
- document reprocessing;
- failed notification retry;
- calendar resynchronization;
- failed job retry;
- data consistency checks;
- orphan detection;
- configuration validation.

Destructive maintenance operations require elevated authorization.

61. System Health Dashboard

System administrators should have access to:

API health

Database health

Message broker health

Search health

Object storage health

Worker health

Scheduler health

External integrations

Queue depth

Failed jobs

Storage usage

System health must not expose confidential case content.

62. Storage Management

Administrators should be able to monitor:

Document storage

Search index storage

Database size

Backup status

Object count

Processing queue

Storage statistics should be available without exposing document contents.

63. Backup Administration

Backup configuration is primarily infrastructure-controlled under v1.1.

The application administration interface may display:

last successful backup

backup age

backup status

restore-test status

It should not implement backup mechanisms itself.

64. Audit Administration

Administrators and auditors should have controlled access to audit logs.

Search criteria:

user

action

resource

case

organization

date range

IP address where appropriate

request ID

Audit logs must be protected from modification by ordinary administrators.

65. Configuration Security

Administrative endpoints must enforce:

- MFA where configured;
- privileged-role authorization;
- session protection;

- rate limiting;
- audit logging;
- CSRF protection where cookie authentication is used;
- secure reauthentication for sensitive actions.

Sensitive operations may require step-up authentication.

66. Break-Glass Administration

Emergency access may be supported.

Break-glass access must:

1. require explicit reason;
2. require elevated authentication;
3. create a high-priority audit event;
4. notify security administrators where configured;
5. expire automatically;
6. be reviewed afterward.

Break-glass must not become normal administrative access.

67. Administrative API

Example endpoints:

Organizations

```
GET    /api/v1/admin/organizations
POST   /api/v1/admin/organizations
GET    /api/v1/admin/organizations/{id}
PUT    /api/v1/admin/organizations/{id}
```

Users

```
GET    /api/v1/admin/users
GET    /api/v1/admin/users/{id}
PUT    /api/v1/admin/users/{id}
POST   /api/v1/admin/users/{id}/suspend
POST   /api/v1/admin/users/{id}/deactivate
```

Teams

```
GET    /api/v1/admin/teams
POST   /api/v1/admin/teams
PUT    /api/v1/admin/teams/{id}
```

Workflows

```
GET    /api/v1/admin/workflows
POST   /api/v1/admin/workflows
POST   /api/v1/admin/workflows/{id}/validate
POST   /api/v1/admin/workflows/{id}/publish
POST   /api/v1/admin/workflows/{id}/retire
```

Configuration

```
GET /api/v1/admin/configuration
PUT /api/v1/admin/configuration
```

Jobs

```
GET    /api/v1/admin/jobs
GET    /api/v1/admin/jobs/{id}
POST   /api/v1/admin/jobs/{id}/retry
POST   /api/v1/admin/jobs/{id}/cancel
```

68. Administrative API Security

Administrative endpoints should use a separate authorization namespace.

Example:

```
admin.users.read
admin.users.manage
admin.roles.manage
admin.workflows.manage
admin.security.manage
admin.integrations.manage
admin.configuration.manage
```

A normal case user must never gain administrative permissions simply because they have access to a case.

69. Configuration Transactions

Configuration changes must use transactional semantics.

For example:

Update workflow
+
Create audit event
+
Create outbox event
must occur within the same transaction where applicable.

70. Event-Driven Configuration

Configuration changes may publish events.

Example:

```
{  
  "event": "workflow.published",  
  "organization_id": "uuid",  
  "workflow_id": "uuid",  
  "workflow_version": 4  
}
```

Events should contain identifiers and metadata rather than confidential case content.

Consumers must be idempotent.

71. Configuration Caching

Frequently accessed configuration may be cached.

Examples:

- workflow definitions;
- case types;
- notification templates;
- classification trees.

Cache invalidation must occur after configuration changes.

The database remains authoritative.

72. Configuration Propagation

After publishing configuration:

Database

↓

Configuration event

↓

Cache invalidation

↓

Workers/services refresh

A configuration change must not depend on an application restart unless explicitly unavoidable.

73. Localization Administration

Organizations may configure:

default language

default timezone

date format

number format

currency

The initial language set should include:

is-IS

en-US / en-GB

where appropriate.

Legal terminology should be separately configurable from ordinary UI translation.

74. Icelandic Legal Terminology

The system should distinguish:

Technical label

Display label

Legal terminology

For example:

Machine:

DEADLINE

English:

Deadline

Icelandic:

Frestur

More specific legal terminology may vary according to context and should be reviewed before production use.

75. Configuration Documentation

Every configurable option should provide:

Name

Description

Scope

Default

Allowed values

Security impact

Dependencies

Version

High-risk configuration should additionally explain:

Potential consequences

76. Administrative Change Preview

Before activation, the UI should provide a preview.

Example:

You are changing:

Deadline Rule:

Civil response deadline

Current:

14 calendar days

New:

14 working days

Affected:

New cases only

Existing cases:

Unchanged

This significantly reduces configuration errors.

77. Configuration Impact Analysis

Where feasible, administrators should be able to determine what a configuration change affects.

Examples:

Workflow v5

Used by:

37 case templates

128 active cases

or:

Deadline rule v3

Used by:

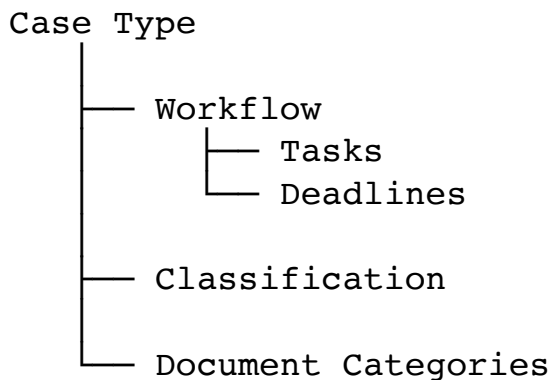
54 active cases

High-impact changes should require explicit confirmation.

78. Configuration Dependency Graph

The administration subsystem should internally understand relationships.

Example:



This prevents deletion of configuration objects that are still referenced.

79. Deletion Policy

Configuration should generally use deactivation rather than physical deletion.

For example:

ACTIVE

↓

INACTIVE

Physical deletion should only be permitted where:

- no historical dependency exists;
- retention requirements permit it;
- authorization permits it;
- audit requirements are satisfied.

80. Administrative Search

Administrators should be able to search configuration objects by:

name
code
type
status
version
owner
created_by
updated_at

Administrative search must also be authorization-aware.

81. Multi-Tenant Considerations

If ILCMS operates as a multi-tenant SaaS platform:

Platform
├── Organization A
├── Organization B
└── Organization C

Organization administrators must never access another organization's configuration.

System administrators should have separate privileges for tenant management.

Tenant configuration should not leak through:

- search;
- reports;
- error messages;
- identifiers;
- logs;
- metrics.

82. Air-Gapped Deployment

All core administration functions must work without Internet connectivity.

External integrations are optional.

An air-gapped installation should support:

Local IdP

Local SMTP

Local object storage

Local PostgreSQL

Local search

Local OCR

Local message broker

Local AI services

Configuration import/export should make it possible to administer environments where no external package repository or cloud control plane is available.

83. Infrastructure Boundary

Application configuration must remain distinct from infrastructure configuration.

Application configuration

Case types

Workflows

Roles

Notifications

Retention

Classification

Infrastructure configuration

Kubernetes

CPU/memory

Storage classes

Ingress

TLS

PostgreSQL clusters

Message broker

Vault

Infrastructure configuration remains under v1.1/IaC/GitOps.

The ILCMS administration UI must not attempt to manage Kubernetes infrastructure.

84. Testing Requirements

Organization

Test:

- organization isolation;
- departments;
- teams;
- membership;
- user lifecycle.

Authorization

Test:

- role assignment;
- permission changes;
- privilege escalation;
- tenant isolation;
- administrative endpoint protection.

Configuration

Test:

- validation;
- versioning;
- publishing;
- rollback;
- dependency validation;
- import/export.

Workflow

Test:

- workflow validation;
- version activation;
- existing-case behavior;
- new-case behavior.

Deadline rules

Test:

- rule calculation;
- calendars;
- weekends;
- holidays;
- versioning;
- overrides.

Retention

Test:

- policy selection;
- legal holds;
- expiration;
- disposition approval.

85. Security Test Scenarios

The security test suite must include attempts to:

User → access another organization's configuration

Case user → call admin endpoint

Team manager → modify security policy

Organization admin → access system-wide configuration

Deactivated user → access administration API

Admin → bypass object-level case authorization

Admin → expose stored integration credentials

All must fail unless explicitly authorized.

86. Performance Requirements

Administrative operations should normally be interactive.

Targets:

Configuration read:

P95 < 500 ms

Simple configuration update:

P95 < 1 second

Workflow validation:

< 5 seconds for normal workflow sizes

Large configuration import:

asynchronous

Large operations must use background jobs.

87. Acceptance Criteria

v1.9 is considered implemented when:

Organization

- Organizations can be configured.
- Departments can be created.
- Teams can be created.
- User membership is manageable.
- User lifecycle is supported.

Authorization

- Roles are managed securely.
- Permissions are configurable within defined boundaries.
- Object authorization remains enforced.
- Tenant isolation is tested.

Case configuration

- Case types are configurable.

- Classification is configurable.
- Case templates are configurable.
- Numbering is configurable.
- Workflow versions are supported.

Work management

- Task templates work.
- Deadline rules work.
- Calendar rules work.
- Notification rules work.

Documents

- Document categories are configurable.
- Retention policies integrate with v1.6.
- Legal holds prevent prohibited disposition.

Administration

- Configuration changes are audited.
- High-impact configuration can require approval.
- Configuration history is preserved.
- Configuration import/export works.
- Dependency validation works.
- Configuration rollback is version-based.
- Integration credentials are protected.
- System health is visible to authorized administrators.

Security

- Administrative APIs are separately permissioned.
- MFA/step-up authentication is supported where configured.
- Break-glass access is audited.
- Administrative operations cannot bypass case authorization.

- Secrets are never exposed through the UI or API.

88. Implementation Phases

Phase 1 — Organization Administration

Implement:

Organizations

Departments

Teams

Users

Memberships

Basic roles

Phase 2 — Master Data

Implement:

Case types

Classifications

Courts

Document categories

Reference data

Numbering

Phase 3 — Workflow Configuration

Implement:

Workflow templates

Workflow versions

Workflow validation

Task templates

Case templates

Phase 4 — Rules and Policies

Implement:

Deadline rules

Calendars

Notification rules

Retention policies

Legal holds

Phase 5 — System Administration

Implement:

System configuration
Feature flags
Integration administration
Job administration
Health dashboard
Audit administration

Phase 6 — Configuration Lifecycle

Implement:

Configuration export
Configuration import
Validation
Promotion
Approval
Impact analysis
Rollback

89. Suggested Database Tables

v1.9 introduces or formalizes:

organizations
departments
teams
team_memberships

users
roles
permissions
role_permissions
user_roles

case_types
classifications
classification_versions

workflow_definitions

workflow_versions
workflow_states
workflow_transitions

case_templates
task_templates
deadline_rules
deadline_rule_versions

calendars
calendar_rules

notification_rules
notification_templates

document_categories
retention_policies
retention_policy_versions
legal_holds

custom_fields
custom_field_definitions

numbering_schemes

system_settings
organization_settings
user_settings

feature_flags

integrations
integration_configurations

configuration_changes
configuration_approvals

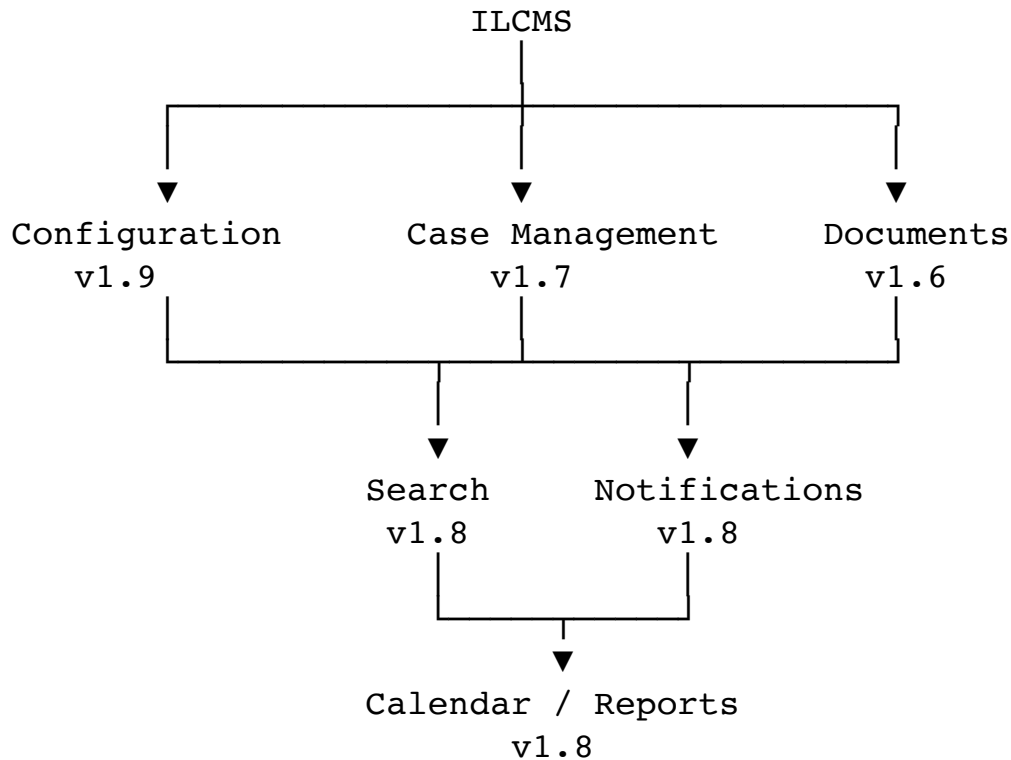
reference_data
courts

administrative_jobs

Existing tables from v1.3–v1.8 should be reused rather than duplicated.

90. Architectural Relationship

The resulting architecture becomes:



Administration provides the configuration layer underneath these capabilities.

91. Final Architecture Principle

The administration system should follow this principle:

Configuration changes system behavior; configuration itself must therefore be treated as controlled, versioned, auditable data.

In particular:

Configuration



Validation



Approval where required



Version



Publish



Audit

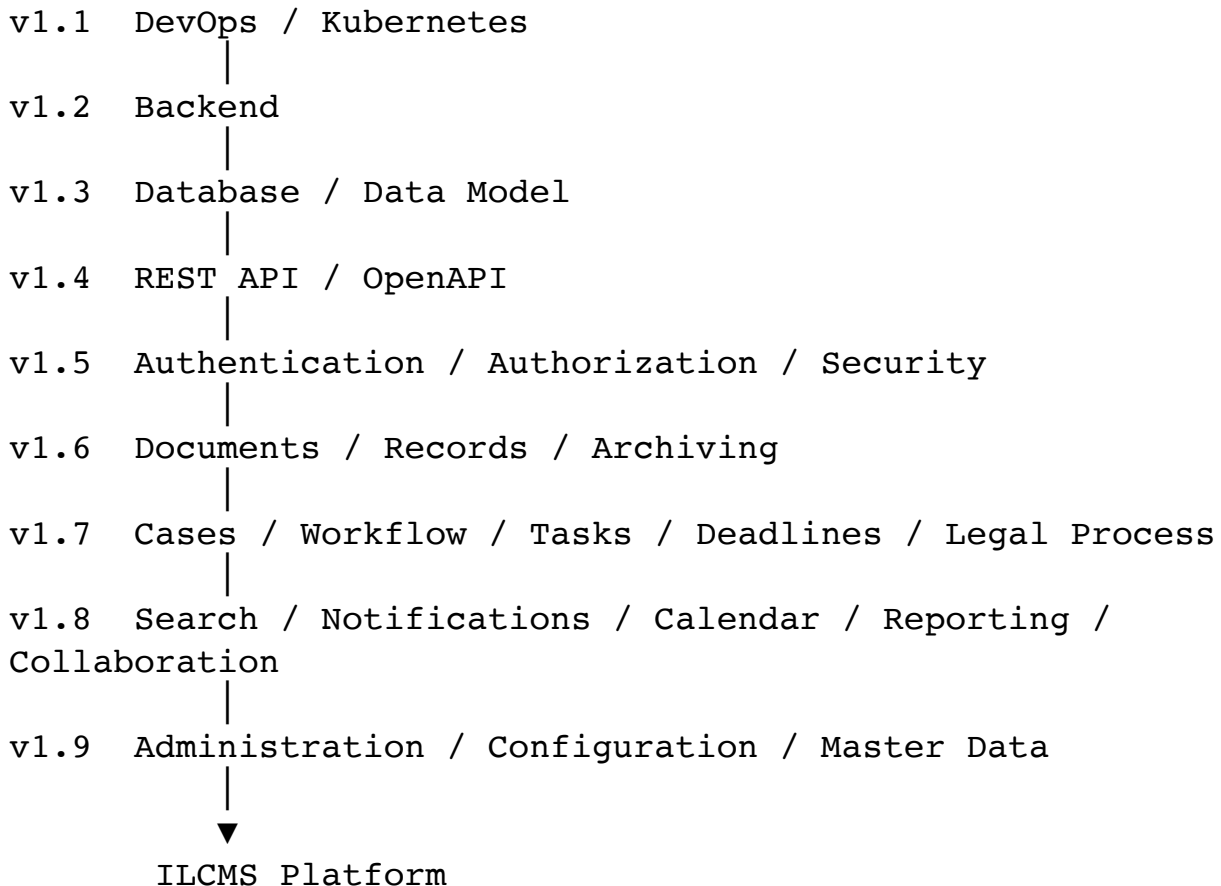


Application

This prevents administrators from accidentally changing the interpretation of historical legal records while still allowing a legal organization to adapt ILCMS to its own practices.

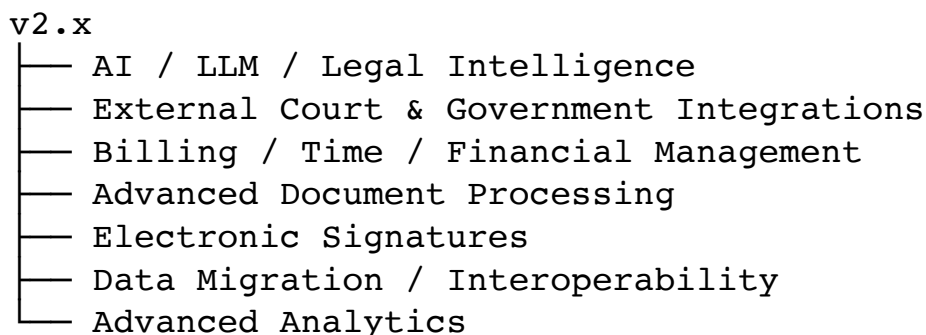
92. Overall ILCMS Architecture After v1.9

The specifications now form a substantially complete platform architecture:



At this point, the core **transactional legal case-management platform** is architecturally defined.

The major remaining functional architecture is therefore likely to be:



These should be treated as extensions to the core platform rather than allowing them to redefine the authoritative case-management model.

The next specification I would recommend is **v2.0 — AI/LLM-Assisted Legal Intelligence, RAG & AI Governance**. Given the direction of ILCMS, this deserves its own major version rather than being buried as another subsystem: it would define local/air-gapped LLMs, RAG over the case/

document repository, citation and source traceability, prompt/model versioning, human approval, hallucination controls, confidentiality boundaries, and AI Act/privacy considerations.